

ChainGrade 虚拟机开发与适配说明

本文档用于帮助新组员在虚拟机中建立与 ChainGrade 项目兼容的开发环境。课程答辩与 CCF 竞赛使用同一个项目、同一仓库和同一套架构。

1. 项目运行形态

ChainGrade 是一个 pnpm TypeScript monorepo，主要由四部分组成：

组件	目录	技术与职责
Web 前端	apps/web	Vue 3、Vite，提供教师、复核员、学生和公开验证者界面
业务 API	apps/api	Fastify、Fabric Gateway SDK，负责鉴权、业务编排和账本访问
共享类型	packages/shared	Zod Schema、领域类型和前后端共享约束
成绩链码	chaincode/grade-contract	TypeScript Fabric Contract，负责凭证、修订、撤销、申诉和有限披露状态

完整开发环境还包含 Hyperledger Fabric 网络：一个 Raft orderer、两个 Peer、LevelDB 状态数据库和一份 Node.js Chaincode-as-a-Service 链码。

2. 推荐虚拟机基线

项目	推荐值	最低可用值	说明
操作系统	Ubuntu 22.04 LTS	兼容的现代 x86_64 Linux	项目脚本按 Bash/Linux 编写
CPU 架构	x86_64 / amd64	x86_64 / amd64	bootstrap.sh 当前明确不支持 ARM64
CPU	4 vCPU	2 vCPU	2 核可运行，但 Fabric 部署和 TypeScript 构建较慢
内存	8 GiB	4 GiB	只做前端/API 时 4 GiB 通常足够；完整 Fabric 建议 8 GiB
虚拟磁盘	40 GiB	至少 20 GiB 可用空间	原生预检硬性要求项目所在分区剩余不少于 10 GiB
GPU	不需要	不需要	项目不依赖 CUDA 或模型训练
网络	可访问 GitHub、Node/npm 和 Docker Hub	能获取固定版本依赖	第一次初始化需要下载 Fabric 二进制和容器镜像

当前学校服务器实测环境为 Ubuntu 22.04、Linux 6.8 x86_64、Node.js 24.19.0、pnpm 11.19.0、Fabric 2.5.16。服务器的 96 核 CPU 和 251 GiB 内存属于共享服务器资源，不是项目最低硬件要求。

3. 固定软件版本

为减少“我的机器能运行、组员机器不能运行”的情况，建议采用以下版本：

软件	项目版本
Node.js	24.19.0; 仓库最低要求为 >=22
pnpm	11.19.0
Hyperledger Fabric	2.5.16 LTS
Fabric CA	1.5.17
Fabric samples	commit 05edea01d4cf24dd4087bd3750c36e690dc4d6ff
jq	1.7.1
Docker Engine	当前已验证环境为 28.1.1
Docker Compose	当前已验证环境为 Compose v2 2.35.1

仓库中的主要应用版本包括 Vue 3.5.41、Vite 8.2.2、TypeScript 5.9.3、Fastify 5.12.1、Fabric Gateway SDK 1.12.0，以及 Fabric Contract/Shim 2.5.8。正常开发不需要 GPU、Java、Python、PostgreSQL 或本地 Go 编译器。

4. 基础系统工具

Ubuntu 虚拟机至少需要以下工具：

```
sudo apt update
sudo apt install -y \
  git curl ca-certificates tar xz-utils \
  openssl iproute2
```

其中 `iproute2` 提供预检和启停脚本使用的 `ss` 命令。Docker Engine 与 Compose v2 建议按照 Docker 官方 Ubuntu 安装方法配置，并确保当前用户能够执行：

```
docker version
docker compose version
```

不要在多人共享服务器上执行 `docker system prune`、删除 Docker data-root 或重启共享 Docker 服务；个人虚拟机中的 Docker 则由该组员自行管理。

5. Node.js 与 pnpm

安装 Node.js 24.19.0 后检查：

```
node --version
corepack --version
```

启用项目锁定的 pnpm：

```
corepack enable
corepack prepare pnpm@11.19.0 --activate
pnpm --version
```

期望 `pnpm --version` 输出 `11.19.0`。

项目普通开发命令可以使用系统或版本管理器中的 Node。`infra/demo/preview.sh` 为了答辩稳定性，会显式寻找 `.tools/node/bin/node` 和 `.tools/node/bin/corepack`。如果组员需要使用该一键演示脚本，应把 Node 24.19.0 解压到项目的 `.tools/node`，或让 `.tools/node` 指向实际 Node 安装根目录；只执行 `pnpm dev:*` 时不要求这一目录。

6. 克隆与基础验证

```
git clone https://github.com/ytq0198/Echoes-of-the-Chain.git
cd Echoes-of-the-Chain
```

```
pnpm install --frozen-lockfile
pnpm check
pnpm test
pnpm build
```

四条命令全部通过，才能认为 Node、pnpm、锁文件和工作区链接已经适配。不要删除或重新生成 `pnpm-lock.yaml` 来绕过安装问题。

7. 三种开发模式

7.1 模式 A：完整 Docker Fabric 开发（新虚拟机推荐）

全新虚拟机应优先使用这一方式，因为脚本会创建 Fabric CA、MSP/TLS 材料、通道和三类开发身份。

```
./infra/fabric/bootstrap.sh
./infra/fabric/pull-images.sh
./infra/fabric/network.sh up

CHAINCODE_VERSION=0.9 \
CHAINCODE_SEQUENCE=1 \
./infra/fabric/network.sh deploy

./infra/fabric/network.sh status
```

启动真实 Fabric API:

```
FABRIC_ENABLED=true \
CHAINGRADE_PROJECT_ROOT="$PWD" \
pnpm dev:api
```

另开一个终端启动前端:

```
pnpm dev:web
```

浏览器访问:

```
http://127.0.0.1:5173/
```

7.2 模式 B：只做前端或普通 API

如果当前任务是 UI、交互、响应式布局或不依赖真实账本的 API，可以不启动 Docker/Fabric:

另开终端：

```
pnpm dev:web
```

该模式使用明确标注的进程内演示账本，适合开发和浏览器验收，但不会产生真实 Fabric transaction ID，不能把结果写成真实链上证据。

7.3 模式 C：无 Docker 原生 Fabric

学校服务器目前使用这一模式运行 Fabric 2.5.16：

```
orderer 127.0.0.1:7050
├─ peer0.org1 127.0.0.1:7051
├─ peer0.org2 127.0.0.1:9051
└─ grade Node.js CCAAS 127.0.0.1:9999
```

原生模式依赖已经生成的通道区块、MSP/TLS 证书和应用身份，而这些敏感运行材料不会提交到 Git。因此，只有仓库代码的全新虚拟机不能直接执行原生启动。新环境应先使用模式 A 创建材料，或者使用经过授权、校验和安全传输的专用恢复包。

已有完整材料时，相关命令为：

```
./infra/fabric-native/preflight.sh
./infra/fabric-native/native-network.sh up
./infra/fabric-native/deploy-chaincode.sh deploy
./infra/fabric-native/native-network.sh status
./infra/fabric-native/deploy-chaincode.sh status
```

8. 端口要求

常用端口如下。启动前应避免其他程序占用：

端口	用途
3000	Fastify API
5173	Vite Web
7050	Raft orderer
7051 / 7052	Org1 Peer / 链码端口
7053	Orderer Admin
9051 / 9052	Org2 Peer / 链码端口
9443–9445	Fabric 运维/监控端口
9999	Node.js CCAAS 链码

Docker Fabric 还会使用测试网络的 CA 端口，例如 7054、8054 和 9054。个人虚拟机通常不需要向局域网公开这些端口。

9. 从宿主机访问虚拟机界面

Vite 和 API 默认都只监听虚拟机内的 `127.0.0.1`。如果组员在宿主机浏览器中访问，可以使用 SSH 隧道：

```
ssh -N -L 5173:127.0.0.1:5173 <虚拟机用户>@<虚拟机地址>
```

然后在宿主机访问 `http://127.0.0.1:5173/`。只需转发 5173，因为 `/api` 请求会由虚拟机中的 Vite 同源代理转发到 `127.0.0.1:3000`。

也可以在可信的仅主机网络中临时让 Vite 监听 `0.0.0.0`，但不要在公共网络中直接暴露开发服务器和 Fabric 端口。

10. 鉴权环境变量

默认开发可以关闭鉴权。需要验证教师、复核员和学生登录时，从 `.env.example` 复制变量到私有 shell 环境并替换占位密码：

```
export AUTH_ENABLED=true
export AUTH_SESSION_SECRET='<至少 32 个字符的随机值>'
export AUTH_ISSUER_USERNAME='demo-issuer'
export AUTH_ISSUER_PASSWORD='<私有密码>'
export AUTH_REVIEWER_USERNAME='demo-reviewer'
export AUTH_REVIEWER_PASSWORD='<私有密码>'
export AUTH_STUDENT_USERNAME='demo-student'
export AUTH_STUDENT_PASSWORD='<私有密码>'
export AUTH_ALLOWED_ORIGINS='http://127.0.0.1:5173'
export AUTH_SECURE_COOKIE=false
```

真实密码、会话密钥、MSP 私钥、恢复包和 `.runtime` 目录不得提交 Git。正式 HTTPS 环境必须设置 `AUTH_SECURE_COOKIE=true`。

11. 最终适配检查清单

组员在开始具体任务前，应把以下结果发回组内：

```
uname -m
node --version
pnpm --version
docker version
docker compose version
df -h .

pnpm install --frozen-lockfile
pnpm check
pnpm test
pnpm build
```

如果负责真实链码或 API/Fabric 集成，还需提供：

```
./infra/fabric/network.sh status
curl --fail http://127.0.0.1:3000/api/v1/meta
```

期望架构为 `x86_64`，Node.js 为 24.19.0，pnpm 为 11.19.0，类型检查、测试和生产构建全部通过。元数据接口在真实网络模式下应显示 `ledgerMode=fabric`。

12. 常见不兼容问题

现象	常见原因	处理方式
Unsupported architecture	虚拟机是 ARM64	改用 x86_64 Ubuntu 虚拟机
pnpm 版本不同或锁文件变化	没有通过 Corepack 固定版本	激活 pnpm 11.19.0，恢复锁文件后重新安装
Fabric samples missing	没有运行 bootstrap	执行 <code>./infra/fabric/bootstrap.sh</code>
Docker permission denied	当前用户无 Docker 权限	正确配置 Docker 用户权限，重新登录会话
端口已占用	其他服务占用 3000、5173 或 Fabric 端口	停止冲突程序或调整独立虚拟机端口规划
原生预检缺证书/通道块	新克隆仓库没有 Git 外运行材料	先使用 Docker 模式初始化，不要伪造或提交私钥
Vite ENOSPC	宿主或共享系统 watcher 数量耗尽	对 Vite 使用 polling，不要擅自修改共享内核参数
页面能打开但业务接口不可用	API 未选择账本模式	设置 <code>FABRIC_ENABLED=true</code> 或明确使用 <code>DEMO_ENABLED=true</code>

13. 仓库中的权威配置来源

- 根目录 `package.json`：Node.js 与 pnpm 版本、工作区命令。
- `pnpm-lock.yaml`：完整依赖锁定结果。
- `infra/fabric/versions.env`：Fabric、Fabric CA、samples 和 jq 固定版本。
- `infra/fabric/README.md`：Docker Fabric 初始化与部署方式。
- `infra/fabric-native/README.md`：无 Docker 原生 Fabric、备份与恢复边界。
- `.env.example`：鉴权变量模板。
- `deliverables/demo/runbook.md`：答辩环境启停与 SSH 隧道方式。